

University of Wah

Department of Computer Science



Submit to:

Ms. Muniba Khan

Submit by:

Muhammad Talha(101)
Muhammad Rafay Ali(063)

Course:

Data Structure and Algorithm_Lab

Reg No:

UW-24-CS-BS-101
UW-24-CS-BS-063

Section:

BS-CS-3rd-B

Open Ended Lab

Task A

1) How Priority Queues differ from normal queues.

Priority Queue is different from normal Queue because priority Queue execute based on the priority given to a Queue. On the other hand normal Queue just execute in normal way Like (FIFO) concept.

This makes both Queue different from each other.

Reference: Greek For Greek website

2) Why tree-based implementations (heaps, BSTs) are preferred over arrays in large-scale systems.

Tree based implementation is preferred over array because Array is in fixed size, Insertion or Deletion have time complexity $O(n)$, Searching and reordering is difficult and poor scalability.

On the other hand tree didn't have these problems. (heap) They have priority order, time complexity is $O(\log n)$.

(BST) They have sorted system, time complexity is $O(\log n)$, and have efficient searching.

Preferred: Tree because they have dynamic data handling, faster searching.

Reference: Greek For Greek website

Task B

Priority Queue

```
#include <iostream>
using namespace std;

struct Event {
    int id;
    int priority;
    Event* next;
};

Event* front = NULL;

void insertEvent(int id, int priority) {
    Event* newEvent = new Event();
    newEvent->id = id;
    newEvent->priority = priority;
    newEvent->next = NULL;

    if (front == NULL || priority < front->priority) {
        newEvent->next = front;
        front = newEvent;
    } else {
        Event* temp = front;
        while (temp->next != NULL && temp->next->priority <= priority) {
            temp = temp->next;
        }
        newEvent->next = temp->next;
        temp->next = newEvent;
    }
}
```

```

Event* removeEvent() {
    if (front == NULL)
        return NULL;

    Event* temp = front;
    front = front->next;
    return temp;
}

```

Tree

```

struct Node {
    int data;
    Node* left;
    Node* right;
};

Node* createNode(int value) {
    Node* n = new Node();
    n->data = value;
    n->left = NULL;
    n->right = NULL;
    return n;
}

Node* insertBST(Node* root, int value) {
    if (root == NULL)
        return createNode(value);

    if (value < root->data)
        root->left = insertBST(root->left, value);
    else
        root->right = insertBST(root->right, value);

    return root;
}

```

```

int findMin(Node* root) {
    while (root->left != NULL)
        root = root->left;
    return root->data;
}

int findMax(Node* root) {
    while (root->right != NULL)
        root = root->right;
    return root->data;
}

```

```

int main() {

    insertEvent(50, 3);
    insertEvent(30, 1);
    insertEvent(70, 4);
    insertEvent(20, 2);
    insertEvent(90, 5);
    insertEvent(10, 1);
    insertEvent(60, 2);
    insertEvent(40, 3);

    Node* root = NULL;

    cout << "Processing events by priority:\n";

    while (front != NULL) {
        Event* e = removeEvent();
        cout << "Processed Event ID: " << e->id << endl;
        root = insertBST(root, e->id);
    }

    cout << "\nMinimum Event ID in BST: " << findMin(root) << endl;
    cout << "Maximum Event ID in BST: " << findMax(root) << endl;

    return 0;
}

```

Output

Linked List

```

Processing events by priority:
Processed Event ID: 30
Processed Event ID: 10
Processed Event ID: 20
Processed Event ID: 60
Processed Event ID: 50
Processed Event ID: 40
Processed Event ID: 70
Processed Event ID: 90

```

Tree

```

Minimum Event ID in BST: 10
Maximum Event ID in BST: 90

```

Task C

1) Implementation

This Priority Queue and Tree are implemented above both contain

2) Test

a) Random Value

```
// Random input (simple values)
insertEvent(45, 3);
insertEvent(12, 1);
insertEvent(78, 4);
insertEvent(23, 2);
insertEvent(90, 5);
insertEvent(34, 3);
insertEvent(67, 2);
insertEvent(10, 1);

while (front != NULL) {
    Event* e = removeEvent();
    cout << "Processed ID: " << e->id << endl;
    root = insertBST(root, e->id);
}

cout << "Min ID: " << findMin(root) << endl;
cout << "Max ID: " << findMax(root) << endl;
```

```
RANDOM INPUT
Processed ID: 12
Processed ID: 10
Processed ID: 23
Processed ID: 67
Processed ID: 45
Processed ID: 34
Processed ID: 78
Processed ID: 90
Min ID: 10
Max ID: 90
```

b) Sorted Value

```

cout << "\nSORTED INPUT\n";

front = NULL;
root = NULL;

insertEvent(10, 1);
insertEvent(20, 2);
insertEvent(30, 3);
insertEvent(40, 4);
insertEvent(50, 5);
insertEvent(60, 6);
insertEvent(70, 7);
insertEvent(80, 8);

while (front != NULL) {
    Event* e = removeEvent();
    cout << "Processed ID: " << e->id << endl;
    root = insertBST(root, e->id);
}

cout << "Min ID: " << findMin(root) << endl;
cout << "Max ID: " << findMax(root) << endl;

```

Output

```

SORTED INPUT
Processed ID: 10
Processed ID: 20
Processed ID: 30
Processed ID: 40
Processed ID: 50
Processed ID: 60
Processed ID: 70
Processed ID: 80
Min ID: 10
Max ID: 80

```

3) Analyze

a)

Array : $O(n)$

Linked List : $O(\log n)$

b)

In tree:

Slow Searching, High CPU usage, Unsafe for real-time systems.

Array-Based Priority Queue

Deletion is slow, Not suitable for **large systems**, Can delay critical events

No Error Handling

Overflow, System instability, Safety risks in real-time applications

Reference

Greek For Greek Web Site
W3 School Web Site
Google (main Source)